

AI Receipt Processing System

Project

Timeline: 8 Days

Solo Developer

Stack: FastAPI + Next.js
14

AI: Gemini 1.5 Flash

1. What We're Building

A web-based system where users upload receipt images or PDFs. Gemini Vision extracts line items, the backend matches them against a centralized product database, and the system outputs a structured, shareable Google Doc report — eliminating manual data entry and reducing accounting errors.

Architecture Flow



2. Tech Stack

Layer	Choice	Reason
Frontend	Next.js 14 (App Router)	Your stack, good practice ground
Backend	FastAPI	Your stack, async-first
Database	PostgreSQL + SQLAlchemy 2.0	Your stack, reliable
AI / OCR	Gemini 1.5 Flash (vision)	Fast, cheap, multimodal
Output	Google Docs API	Structured, shareable reports
File Storage	Cloudflare R2	Your infra, S3-compatible
Auth	JWT (access + refresh)	Matches your backend pattern
Hosting	Google Cloud Run + Vercel	Your existing setup

3. APIs & Integrations

Gemini 1.5 Flash — Primary AI

- Model: gemini-1.5-flash (vision capable, fast, low cost)

- Usage: Send receipt image → receive structured JSON of line items
- SDK: google-generativeai (Python)
- Prompt instructs model to extract: item_name, barcode, quantity, unit_price, total
- PDF inputs: convert page 1 to image via pdf2image before sending

Gemini Prompt Structure

```
You are an OCR assistant. Extract all line items from this receipt image.
Return ONLY a JSON array, no markdown, no preamble.
Each object: {"item_name": "", "barcode": "", "quantity": 0, "unit_price": 0.0, "total": 0.0}
If a field is not visible, use null.
```

Google Docs API — Report Generation

- Creates a new Google Doc per processed receipt
- Writes: header, full items table, summary section, grand total
- Sets sharing to 'anyone with link can view'
- Auth: Service Account (server-to-server, no OAuth flow needed)
- Also requires Google Drive API to manage sharing permissions

4. Database Schema

Table: products

```
id, barcode VARCHAR UNIQUE
name, category
unit_price DECIMAL, unit
supplier, created_at
```

Table: receipts

```
id, uploaded_by
image_url (R2 path)
status ENUM(pending / processing / done / failed)
raw_extraction JSONB – Gemini raw output
created_at
```

Table: receipt_items

```
id, receipt_id FK, product_id FK NULLABLE
extracted_name, extracted_barcode
extracted_qty, extracted_unit_price
matched BOOLEAN, match_confidence FLOAT
doc_url
```

5. Matching Engine Logic

The matching engine is the most critical component. For each item extracted by Gemini, it runs a priority cascade:

Priority	Strategy	Confidence
1	Exact barcode match	1.00
2	Exact name match (case-insensitive)	0.95
3	Fuzzy name match (rapidfuzz)	Library score
4	No match — flagged for review	< 0.75

Items with confidence below 0.75 are marked as Unmatched and highlighted in the UI for manual review. A manual match dropdown allows the user to link them to the correct product.

6. Google Docs Report Format

Each generated document contains the following sections:

- Header: Receipt date, uploaded by, processing timestamp
- Items Table: Item # | Product Name | Barcode | Qty | Unit Price | Total | Match Status
- Summary: Total items, matched count, unmatched count, grand total
- Footer: Generated by AI Receipt System

7. Backend Project Structure

```
app/
├── api/
│   ├── products.py      # CRUD for product DB
│   ├── receipts.py      # Upload, status, results
│   └── reports.py        # Trigger / get doc
├── services/
│   ├── extraction.py     # Gemini Vision call + parse
│   ├── matching.py       # Fuzzy match + barcode lookup
│   └── docs.py            # Google Docs generation
├── models/
├── schemas/
├── db/
└── core/
    └── config.py         # API keys, settings
```

8. Python Dependencies

Package	Purpose
fastapi, uvicorn	Backend framework
sqlalchemy, alembic, asyncpg	ORM + migrations + async driver
google-generativeai	Gemini API SDK
google-api-python-client	Google Docs + Drive
google-auth	Service account authentication

boto3	Cloudflare R2 (S3-compatible)
rapidfuzz	Fuzzy product name matching
pdf2image	Convert PDF page to image for Gemini
pillow	Image processing
python-multipart	File upload handling in FastAPI
pydantic-settings	Environment config

9. 8-Day Execution Plan

Day 1 Foundation & Setup

Milestone: Both projects boot, DB migrations run, R2 connected

- Init FastAPI project (clean architecture structure)
- Init Next.js 14 project
- PostgreSQL setup + Alembic migrations (all 3 tables)
- Configure .env: Gemini key, Google Service Account, DB URL, R2 credentials
- Set up Cloudflare R2 bucket for receipt storage
- Git repo, branch structure done

Day 2 Product Database

Milestone: Product CRUD working via API and UI

- products CRUD endpoints (POST, GET, GET by id/barcode, PATCH, DELETE)
- Pydantic schemas for product
- Seeding script with 20–30 sample products for testing
- Frontend: Product management page (list + add product form)

Day 3 Gemini Extraction Pipeline

Milestone: Upload receipt → Gemini extracts items → stored in DB

- Receipt upload endpoint → save to R2 → create receipt row
- extraction.py: download from R2, send to Gemini, parse JSON response
- Handle PDF inputs: convert page 1 to image using pdf2image
- Store raw Gemini output in receipts.raw_extraction (JSONB)
- Status tracking: pending → processing → done / failed

Day 4

Matching Engine

Milestone: *Full extraction-to-match pipeline working end-to-end*

- matching.py: priority cascade (barcode → exact name → fuzzy via rapidfuzz)
- Save receipt_items rows with match results and confidence score
- Flag items below 0.75 confidence as unmatched
- Endpoint: GET /receipts/{id}/items — matched + unmatched
- Frontend: Receipt detail page with match badges

Day 5

Google Docs Report

Milestone: *Click button → get shareable Google Doc*

- Service account setup + google-api-python-client integration
- docs.py: create Doc, write header + table + summary, set sharing
- POST /receipts/{id}/generate-report endpoint
- Store doc URL in receipt row
- Test with real receipts end-to-end

Day 6

Frontend Core UI

Milestone: *Upload flow working, clean UI*

- Layout: sidebar nav (Dashboard, Receipts, Products, Reports)
- Dashboard: stats cards (total receipts, matched %, recent activity)
- Receipts list page: status chips (pending / processing / done / failed)
- Receipt upload: drag-and-drop zone, progress indicator

Day 7

Frontend Results & Reports

Milestone: *Full user journey works start to finish*

- Receipt detail page: extracted items table with match status
- Unmatched items highlighted in amber/red
- Generate Report button → opens Google Doc in new tab
- Manual match UI: dropdown to link unmatched items to products
- Re-run extraction button for failed receipts

Day 8

Polish & Deploy

Milestone: Live deployed system

- Backend: error handling, logging, Gemini timeout retry
- Edge cases: blurry image, empty extraction, multi-page PDF (page 1 only)
- Background task for extraction using FastAPI BackgroundTasks
- Deploy FastAPI to Google Cloud Run
- Deploy Next.js to Vercel
- Final smoke test with 5 real receipts

10. Key Risks & Mitigations

Risk	Mitigation
Gemini misreads blurry receipt	Validate output shape, flag low-confidence receipts in UI
Barcode not visible on receipt	Fall through to fuzzy name matching automatically
Gemini returns invalid JSON	Try/catch, retry once with stricter prompt
Google Docs API quota hit	Cache doc URL — never regenerate if already exists
PDF multi-page	Process page 1 only, display limitation note in UI
Google Service Account auth issues	Allocate extra time Day 5 — credential setup is complex

This document was generated as a project specification. Day 3 (Gemini extraction) and Day 5 (Google auth) are the highest-risk days — prioritize these during your early morning sessions.